

Vibe coding overview

This video will teach you how to "vibe code". Essentially what I mean is developing something functional in some language and testing its functionality. Lets start off with GPT giving us a break down of this from a technical perspective.

From a technical perspective, **"vibe coding"** can be described as an AI-assisted development workflow where you focus primarily on **desired behavior and outcomes**, while an AI model handles much of the implementation detail.

A useful way to think about it is:

Intent → Generate → Execute → Observe → Debug → Refine → Repeat

You start by defining what the software should do rather than necessarily specifying exactly how every function, class, or algorithm should be implemented. An AI coding model translates that intent into source code in an appropriate language or framework.

The important part is that **generated code is still just unverified source code**. The actual development loop begins when you execute it.

Suppose you tell an AI:

Build a Python program that takes a directory of videos, determines their resolutions, and separates vertical and horizontal videos.

The model might generate code using Python, `pathlib`, and FFmpeg/ffprobe. At that point, you don't necessarily need to manually reason through every implementation detail before trying it. You run it and inspect the observable behavior.

Technically, you're performing an iterative feedback loop:



This is different from simply telling an AI **"write my program."** Effective vibe coding still requires verification. You need some mechanism for determining whether the generated implementation

actually satisfies your requirements.

That verification can range from very informal:

```
Run program
↓
Click button
↓
Did it work?
```

to much more rigorous:

```
Generate implementation
↓
Compile/build
↓
Unit tests
↓
Integration tests
↓
Static analysis
↓
Performance tests
↓
Regression tests
↓
Deploy
```

The interesting technical shift is that **your abstraction level moves upward**. Instead of spending most of your time translating algorithms into syntax yourself, you're increasingly defining requirements, designing system behavior, providing constraints, evaluating output, and debugging failures.

In traditional development, the human often does:

Requirement → Design → Implementation → Testing

With vibe coding, it becomes closer to:

Requirement → AI implementation → Human/automated verification → AI correction

So the AI can own a large portion of the *implementation loop*, while you remain responsible for determining whether the resulting system is actually correct.

And that's the key distinction for the video: **vibe coding isn't inherently “write random prompts until something looks right.”** You can treat it as a legitimate iterative software-development process—as long as the *testing and verification loop* is strong enough to catch when the AI confidently builds the wrong thing.

So essentially to start out with this I would like to vibe code a simple script in python which generates different realistic datasets that I can then later run a suite of automated unit or integration tests against. Ideally with this dataset we produce it has some sort of value inherently, so I would say like a script which allows development of other source code files in python, so essentially a python file code generator. It should be capable of developing functions, classes, imports, logic, etc all in an easy to utilize CLI driven tool. For instance, the tool asks me how much of a specific thing I would like to generate and then goes and creates those files, etc. Ideally this allows for multi file generation with easeability into actual templated project creation.

Show moreShow less

Yes — that is a much better starting project than generating arbitrary fake datasets, because the generated output itself becomes the **dataset**.

What you are effectively describing is a **synthetic Python project generator**. Instead of generating rows of CSV data, it generates structured source-code artifacts that can later be inspected, parsed, executed, linted, and tested.

The pipeline becomes:

```

CLI configuration
  ↓
Project specification
  ↓
Code generator
  ↓
Synthetic Python projects
  ↓
Functions / Classes / Imports / Logic
  ↓
Generated source files
  ↓
Future test suite
  ↓
AST tests / unit tests / integration tests / linting

```

That gives us something particularly useful for the vibe-coding video because we're creating the thing first, then later building increasingly sophisticated automated tests around it.

For an initial MVP, I'd make the CLI work something like this:

```

$ python generator.py

Python Project Generator

Project name: test_project

How many Python files? 5
Functions per file? 4
Classes per file? 2
Methods per class? 3

Include imports? [Y/n]: y
Include type hints? [Y/n]: y
Include docstrings? [Y/n]: y
Include cross-file imports? [Y/n]: y

Generating...

✓ test_project/
  ✓ main.py
  ✓ module_1.py
  ✓ module_2.py
  ✓ module_3.py
  ✓ module_4.py
  ✓ requirements.txt

Generated:
20 functions
8 classes
24 methods
5 source files

```

The important design decision is that **we don't want it generating complete nonsense**. We want syntactically valid, deterministic-ish Python constructs that represent realistic code.

For example, it could generate:

```
Python
from typing import List

def calculate_average(values: List[float]) -> float:
    """Calculate the average value."""
    if not values:
        return 0.0
    return sum(values) / len(values)

class DataProcessor:
    """Processes numerical datasets."""
    def __init__(self, values: List[float]):
        self.values = values
    def maximum(self) -> float:
        return max(self.values)
    def minimum(self) -> float:
        return min(self.values)
```

And another generated file could actually depend on it:

```
Python
from module_1 import DataProcessor

def analyze_numbers(numbers):
    processor = DataProcessor(numbers)
    return {
        "minimum": processor.minimum(),
        "maximum": processor.maximum(),
    }
```

Now you've generated something that has **structural value**.

Later we can test questions such as:

```
Does every generated file parse?
Does every generated module import successfully?
Are all generated function names unique?
Are type hints valid?
Do cross-module imports resolve?
Can generated classes be instantiated?
Do generated methods execute?
Does generated code pass Ruff?
Does generated code pass mypy?
Does generated code compile?
Does the generated project actually run?
Does regeneration produce structurally different projects?
Can we generate 10 files?
100 files?
```

10,000 files?

What happens with malformed generator configuration?

That gives the project a natural progression from vibe coding into testing.

Version 1

I'd intentionally make our first implementation somewhat simple. Something like this:

```
Python
from pathlib import Path
import random

FUNCTION_NAMES = [
    "calculate_total",
    "calculate_average",
    "filter_values",
    "normalize_values",
    "process_items",
    "transform_data",
    "validate_input",
    "format_output",
]

CLASS_NAMES = [
    "DataProcessor",
    "ValueAnalyzer",
    "ItemManager",
    "DataTransformer",
    "InputValidator",
]

FUNCTION_TEMPLATES = [
    """def {name}(values):
    if not values:
        return 0

    return sum(values)
""",
    """def {name}(values):
    return [value for value in values if value > 0]
""",
    """def {name}(values):
    if not values:
        return []

    maximum = max(values)

    if maximum == 0:
        return values

    return [value / maximum for value in values]
""",
]

CLASS_TEMPLATE = """class {name}:
    def __init__(self, values):
        self.values = values

    def get_values(self):
        return self.values
```

```

def count(self):
    return len(self.values)

def total(self):
    return sum(self.values)
"""

def generate_function():
    name = random.choice(FUNCTION_NAMES)
    template = random.choice(FUNCTION_TEMPLATES)

    return template.format(name=name)

def generate_class():
    name = random.choice(CLASS_NAMES)

    return CLASS_TEMPLATE.format(name=name)

def generate_module(function_count, class_count):
    sections = []

    sections.append("from typing import List\\n")

    for _ in range(function_count):
        sections.append(generate_function())

    for _ in range(class_count):
        sections.append(generate_class())

    return "\\n\\n".join(sections)

def generate_project(
    project_name,
    file_count,
    functions_per_file,
    classes_per_file,
):
    project_path = Path(project_name)

    project_path.mkdir(parents=True, exist_ok=True)

    for index in range(1, file_count + 1):
        module_name = f"module_{index}.py"

        module_path = project_path / module_name

        source_code = generate_module(
            functions_per_file,
            classes_per_file,
        )

        module_path.write_text(source_code)

        print(f"Generated {module_path}")

def main():
    print("\\nPython Project Generator\\n")

    project_name = input(
        "Project name: "
    ).strip()

    file_count = int(
        input("How many Python files? ")
    )

    functions_per_file = int(
        input("Functions per file? ")
    )

```

```

)

classes_per_file = int(
    input("Classes per file? ")
)

generate_project(
    project_name,
    file_count,
    functions_per_file,
    classes_per_file,
)

print("\nGeneration complete.")

if __name__ == "__main__":
    main()

```

Already we could run:

```

Bash
python generator.py

```

and say:

```

Project name: generated_project
How many Python files? 10
Functions per file? 5
Classes per file? 2

```

and get:

```

generated_project/

module_1.py
module_2.py
module_3.py
module_4.py
module_5.py
module_6.py
module_7.py
module_8.py
module_9.py
module_10.py

```

containing generated Python.

But this intentionally exposes several problems.

For example:

```

random.choice(FUNCTION_NAMES)

```

means duplicate function names could appear within the same module.

Likewise:

```

random.choice(CLASS_NAMES)

```

could create duplicate classes.

We're also importing:

```
Python
from typing import List
```

without actually necessarily using `List`.

We haven't validated user input.

We're not checking whether generated Python compiles.

We're not creating package structure.

We're not creating cross-file dependencies.

We're not creating `__init__.py`.

We're not creating `main.py`.

We're not tracking metadata about what we generated.

And that is actually **perfect for the video**.

Because now you've genuinely vibe coded Version 1.

You told GPT:

Make me a Python project generator.

It generated something.

You execute it.

It apparently works.

And then you ask the engineering question:

But how do I know it actually works?

That's where the rest of the project becomes interesting.

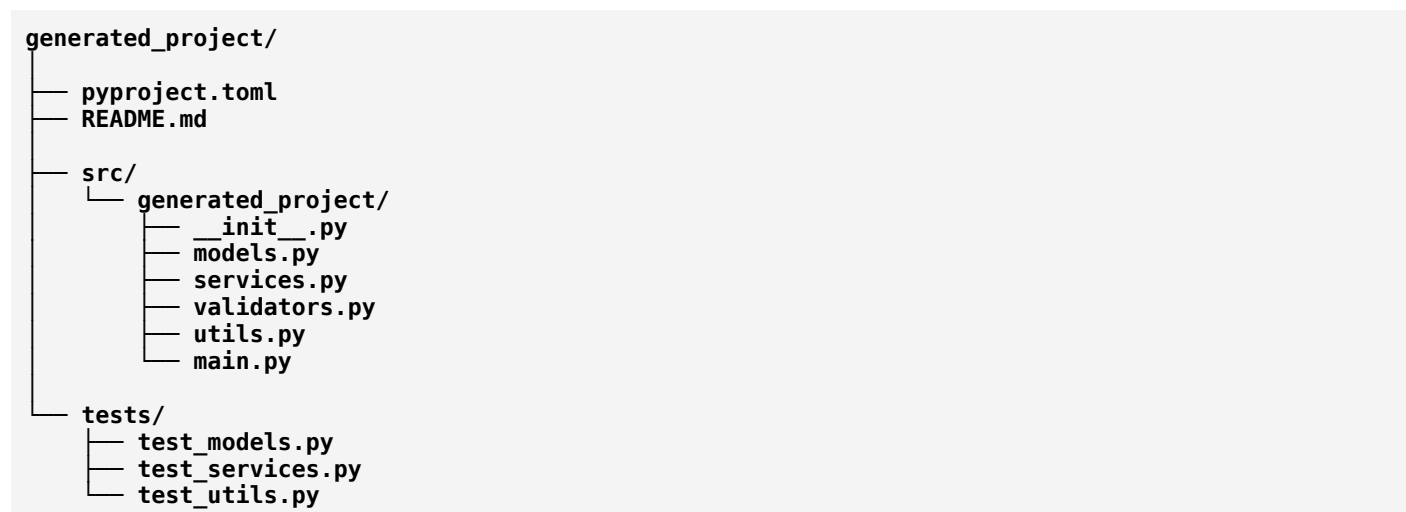
We can progressively turn this from:

```
generator.py
```

into something like:

```
pyforge/
├── generator.py
├── cli.py
├── config.py
├── generators/
│   ├── functions.py
│   ├── classes.py
│   ├── modules.py
│   └── projects.py
├── templates/
│   ├── function_templates.py
│   ├── class_templates.py
│   └── logic_templates.py
└── models/
    └── project_spec.py
```


And generated output could eventually resemble actual projects:



The really cool end goal would be giving the generator a command such as:

```

Bash
python generator.py \
  --projects 100 \
  --files 20 \
  --functions 100 \
  --classes 25
  
```

and producing **100 synthetic Python repositories**.

At that point you haven't just built a toy generator. You've built your own **software-development testing dataset generator**.

That means later in the video/project we can point pytest, AST analysis, linters, static analyzers, CI pipelines, performance tests, code-quality tools, or even another AI coding system at thousands of pieces of source code that **we control and know how they were constructed**.

That makes this a very solid foundation for demonstrating the actual philosophy you're describing:

Vibe code the implementation first. Then progressively engineer confidence into it through automated verification.

Perfect, I would actually like to customize everything I give it, I guess there can be generic defaults if nothing is entered or the user doesn't enter anything but I would like to account for this. Generate me some source code and lets get started.

A good first version is a **single-file CLI generator** that lets you override nearly everything, while still giving sensible defaults when you just hit Enter.

This gives us something immediately runnable, but also leaves plenty of room for later refactoring and testing.

```

Python
from __future__ import annotations

import random
import re
from dataclasses import dataclass
from pathlib import Path
  
```

```

from typing import Optional

DEFAULT_PROJECT_NAME = "generated_project"
DEFAULT_FILE_COUNT = 3
DEFAULT_FUNCTION_COUNT = 3
DEFAULT_CLASS_COUNT = 2
DEFAULT_METHOD_COUNT = 3

DEFAULT_IMPORTS = [
    "from pathlib import Path",
    "from typing import Any",
]

DEFAULT_FUNCTION_PREFIX = "generated_function"
DEFAULT_CLASS_PREFIX = "GeneratedClass"
DEFAULT_METHOD_PREFIX = "generated_method"

@dataclass
class GeneratorConfig:
    project_name: str
    output_directory: Path

    file_count: int
    functions_per_file: int
    classes_per_file: int
    methods_per_class: int

    file_prefix: str
    function_prefix: str
    class_prefix: str
    method_prefix: str

    include_imports: bool
    include_docstrings: bool
    include_type_hints: bool
    include_main: bool
    include_init: bool

    custom_imports: list[str]

    randomize_logic: bool
    random_seed: Optional[int]

def ask_string(prompt: str, default: str) -> str:
    value = input(f"{prompt} [{default}]: ").strip()

    if not value:
        return default

    return value

def ask_int(prompt: str, default: int, minimum: int = 0) -> int:
    while True:
        value = input(f"{prompt} [{default}]: ").strip()

        if not value:
            return default

        try:
            number = int(value)

            if number < minimum:
                print(f"Value must be at least {minimum}.")
                continue

            return number

        except ValueError:
            print("Please enter a valid integer.")

```

```
def ask_bool(prompt: str, default: bool = True) -> bool:
    default_display = "Y/n" if default else "y/N"

    while True:
        value = input(f"{prompt} [{default_display}]: ").strip().lower()

        if not value:
            return default

        if value in {"y", "yes", "true", "1"}:
            return True

        if value in {"n", "no", "false", "0"}:
            return False

        print("Please enter yes or no.")

def sanitize_identifier(value: str, fallback: str) -> str:
    value = re.sub(r"\W+", "_", value)

    if not value:
        return fallback

    if value[0].isdigit():
        value = f"_{value}"

    return value

def sanitize_filename(value: str) -> str:
    value = re.sub(r"[^a-zA-Z0-9_-]", "_", value)

    return value or "module"

def collect_custom_imports() -> list[str]:
    imports = []

    print("\nCustom imports")
    print("Enter one import per line.")
    print("Press Enter on an empty line when finished.\n")

    while True:
        value = input("Import: ").strip()

        if not value:
            break

        imports.append(value)

    return imports

def collect_configuration() -> GeneratorConfig:
    print("=" * 60)
    print("PYTHON SYNTHETIC PROJECT GENERATOR")
    print("=" * 60)

    print("\nPress Enter to accept any default.\n")

    project_name = ask_string(
        "Project name",
        DEFAULT_PROJECT_NAME,
    )

    output_directory_string = ask_string(
        "Output directory",
        "./output",
    )

    file_count = ask_int(
        "Number of Python files",
        DEFAULT_FILE_COUNT,
        minimum=1,
    )
```

```

)

functions_per_file = ask_int(
    "Functions per file",
    DEFAULT_FUNCTION_COUNT,
)

classes_per_file = ask_int(
    "Classes per file",
    DEFAULT_CLASS_COUNT,
)

methods_per_class = ask_int(
    "Methods per class",
    DEFAULT_METHOD_COUNT,
)

file_prefix = ask_string(
    "Generated file prefix",
    "module",
)

function_prefix = ask_string(
    "Function name prefix",
    DEFAULT_FUNCTION_PREFIX,
)

class_prefix = ask_string(
    "Class name prefix",
    DEFAULT_CLASS_PREFIX,
)

method_prefix = ask_string(
    "Method name prefix",
    DEFAULT_METHOD_PREFIX,
)

include_imports = ask_bool(
    "Include imports?",
    True,
)

include_docstrings = ask_bool(
    "Include docstrings?",
    True,
)

include_type_hints = ask_bool(
    "Include type hints?",
    True,
)

include_init = ask_bool(
    "Create __init__.py?",
    True,
)

include_main = ask_bool(
    "Create main.py?",
    True,
)

randomize_logic = ask_bool(
    "Randomize generated function logic?",
    True,
)

random_seed_input = input(
    "Random seed [leave blank for random]: "
).strip()

random_seed = None

if random_seed_input:
    try:

```

```

        random_seed = int(random_seed_input)
    except ValueError:
        print("Invalid random seed. Using random behavior.")

custom_imports = []

if include_imports:
    use_default_imports = ask_bool(
        "Use default imports?",
        True,
    )

    if use_default_imports:
        custom_imports.extend(DEFAULT_IMPORTS)

    add_custom_imports = ask_bool(
        "Add custom imports?",
        False,
    )

    if add_custom_imports:
        custom_imports.extend(
            collect_custom_imports()
        )

return GeneratorConfig(
    project_name=project_name,
    output_directory=Path(output_directory_string),
    file_count=file_count,
    functions_per_file=functions_per_file,
    classes_per_file=classes_per_file,
    methods_per_class=methods_per_class,
    file_prefix=file_prefix,
    function_prefix=function_prefix,
    class_prefix=class_prefix,
    method_prefix=method_prefix,
    include_imports=include_imports,
    include_docstrings=include_docstrings,
    include_type_hints=include_type_hints,
    include_main=include_main,
    include_init=include_init,
    custom_imports=custom_imports,
    randomize_logic=randomize_logic,
    random_seed=random_seed,
)

```

```

FUNCTION_LOGIC_TEMPLATES = [
    """if not values:
        return 0
    return sum(values)""",
    """if not values:
        return 0
    return sum(values) / len(values)""",
    """return len(values)""",
    """return [value for value in values if value > 0]""",
    """return [value * 2 for value in values]""",
    """if not values:
        return None
    return max(values)""",
    """if not values:
        return None
    return min(values)""",
    """return sorted(values)""",
    """return list(reversed(values))""",
    """return list(set(values))""",
]

```

```

]

def generate_function(
    name: str,
    config: GeneratorConfig,
) -> str:
    name = sanitize_identifier(
        name,
        DEFAULT_FUNCTION_PREFIX,
    )

    if config.include_type_hints:
        signature = f"def {name}(values: list[float]) -> object:"
    else:
        signature = f"def {name}(values):"

    lines = [signature]

    if config.include_docstrings:
        lines.append(
            f'        """Automatically generated function: {name}."""'
        )

    if config.randomize_logic:
        logic = random.choice(FUNCTION_LOGIC_TEMPLATES)
    else:
        logic = FUNCTION_LOGIC_TEMPLATES[0]

    for line in logic.splitlines():
        lines.append(f"        {line}")

    return "\n".join(lines)

METHOD_LOGIC_TEMPLATES = [
    "return len(self.values)",
    "return sum(self.values)",
    "return list(self.values)",
    "return sorted(self.values)",
    "return bool(self.values)",
]

def generate_method(
    name: str,
    config: GeneratorConfig,
) -> str:
    name = sanitize_identifier(
        name,
        DEFAULT_METHOD_PREFIX,
    )

    if config.include_type_hints:
        signature = f"def {name}(self) -> object:"
    else:
        signature = f"def {name}(self):"

    lines = [signature]

    if config.include_docstrings:
        lines.append(
            f'        """Automatically generated method: {name}."""'
        )

    logic = random.choice(METHOD_LOGIC_TEMPLATES)

    lines.append(f"        {logic}")

    return "\n".join(lines)

def generate_class(
    name: str,
    method_count: int,

```

```

class_index: int,
config: GeneratorConfig,
) -> str:
    name = sanitize_identifier(
        name,
        DEFAULT_CLASS_PREFIX,
    )

    lines = [f"class {name}:"]

    if config.include_docstrings:
        lines.append(
            f'        """Automatically generated class: {name}."""'
        )

    if config.include_type_hints:
        lines.extend(
            [
                """
                "    def __init__(self, values: list[float]) -> None:",
                "        self.values = values",
            ]
        )
    else:
        lines.extend(
            [
                """
                "    def __init__(self, values):",
                "        self.values = values",
            ]
        )

    for method_index in range(1, method_count + 1):
        method_name = (
            f"{config.method_prefix}_"
            f"{class_index}_"
            f"{method_index}"
        )

        lines.append("")
        lines.append(
            generate_method(
                method_name,
                config,
            )
        )

    return "\n".join(lines)

def generate_module(
    module_index: int,
    config: GeneratorConfig,
) -> str:
    sections = []

    if config.include_imports and config.custom_imports:
        sections.append(
            "\n".join(config.custom_imports)
        )

    for function_index in range(
        1,
        config.functions_per_file + 1,
    ):
        function_name = (
            f"{config.function_prefix}_"
            f"{module_index}_"
            f"{function_index}"
        )

        sections.append(
            generate_function(
                function_name,
                config,
            )
        )

```

```

    )
)

for class_index in range(
    1,
    config.classes_per_file + 1,
):
    class_name = (
        f"{config.class_prefix}"
        f"{module_index}_"
        f"{class_index}"
    )

    sections.append(
        generate_class(
            class_name,
            config.methods_per_class,
            class_index,
            config,
        )
    )

return "\n\n\n".join(sections) + "\n"

def generate_main_file(
    module_names: list[str],
    config: GeneratorConfig,
) -> str:
    lines = []

    if config.include_docstrings:
        lines.append(
            """Automatically generated application entry point."""
        )
        lines.append("")

    if module_names:
        lines.append(
            f"import {module_names[0]}"
        )
        lines.append("")

    lines.extend(
        [
            """
            def main():
                print("Generated project is running.")
            """,
            """
            if __name__ == "__main__":
                main()
            """,
        ]
    )

    return "\n".join(lines)

def generate_project(
    config: GeneratorConfig,
) -> Path:
    if config.random_seed is not None:
        random.seed(config.random_seed)

    safe_project_name = sanitize_filename(
        config.project_name
    )

    project_directory = (
        config.output_directory
        / safe_project_name
    )

    project_directory.mkdir(

```



```

        parents=True,
        exist_ok=True,
    )

    print("\n" + "=" * 60)
    print("GENERATING PROJECT")
    print("=" * 60)

    module_names = []

    for module_index in range(
        1,
        config.file_count + 1,
    ):
        module_name = (
            f"{sanitize_filename(config.file_prefix)}_"
            f"{module_index}"
        )

        module_names.append(module_name)

        module_path = (
            project_directory
            / f"{module_name}.py"
        )

        source_code = generate_module(
            module_index,
            config,
        )

        module_path.write_text(
            source_code,
            encoding="utf-8",
        )

        print(f"Created: {module_path}")

    if config.include_init:
        init_path = (
            project_directory
            / "__init__.py"
        )

        init_path.write_text(
            """"Generated Python package.""""\n",
            encoding="utf-8",
        )

        print(f"Created: {init_path}")

    if config.include_main:
        main_path = (
            project_directory
            / "main.py"
        )

        main_path.write_text(
            generate_main_file(
                module_names,
                config,
            ),
            encoding="utf-8",
        )

        print(f"Created: {main_path}")

    return project_directory

def print_summary(
    config: GeneratorConfig,
    project_directory: Path,
) -> None:
    total_functions = (

```

```

        config.file_count
        * config.functions_per_file
    )

    total_classes = (
        config.file_count
        * config.classes_per_file
    )

    total_methods = (
        total_classes
        * config.methods_per_class
    )

    print("\n" + "=" * 60)
    print("GENERATION COMPLETE")
    print("=" * 60)

    print(
        f"""
Project:
    {config.project_name}

Location:
    {project_directory}

Generated modules:
    {config.file_count}

Generated functions:
    {total_functions}

Generated classes:
    {total_classes}

Generated methods:
    {total_methods}

Random seed:
    {config.random_seed}
"""
    )

def main() -> None:
    config = collect_configuration()

    project_directory = generate_project(
        config
    )

    print_summary(
        config,
        project_directory,
    )

if __name__ == "__main__":
    main()

```

Save that as:

```
generator.py
```

Then run:

```

Bash
python generator.py

```

The interaction will look roughly like:

```
=====
PYTHON SYNTHETIC PROJECT GENERATOR
=====

Press Enter to accept any default.

Project name [generated_project]:
Output directory [./output]:
Number of Python files [3]: 10
Functions per file [3]: 5
Classes per file [2]: 3
Methods per class [3]: 4

Generated file prefix [module]:
Function name prefix [generated_function]: calculate
Class name prefix [GeneratedClass]: Processor
Method name prefix [generated_method]: process

Include imports? [Y/n]:
Include docstrings? [Y/n]:
Include type hints? [Y/n]:
Create __init__.py? [Y/n]:
Create main.py? [Y/n]:
Randomize generated function logic? [Y/n]:

Random seed [leave blank for random]:

Use default imports? [Y/n]:
Add custom imports? [y/N]:
```

If you just spam Enter through everything, you still get a complete project.

If you customize it, you could produce:

```
output/
├── analytics_project/
│   ├── __init__.py
│   ├── main.py
│   ├── analytics_1.py
│   ├── analytics_2.py
│   ├── analytics_3.py
│   ├── analytics_4.py
│   └── analytics_5.py
```

with code such as:

```
Python
from pathlib import Path
from typing import Any

def calculate_1_1(values: list[float]) -> object:
    """Automatically generated function: calculate_1_1."""
    return [value * 2 for value in values]

def calculate_1_2(values: list[float]) -> object:
    """Automatically generated function: calculate_1_2."""
    if not values:
        return None
    return max(values)

class Processor1_1:
    """Automatically generated class: Processor1_1."""

    def __init__(self, values: list[float]) -> None:
```

```

    self.values = values

def process_1_1(self) -> object:
    """Automatically generated method: process_1_1."""
    return sorted(self.values)

def process_1_2(self) -> object:
    """Automatically generated method: process_1_2."""
    return sum(self.values)

```

One thing I intentionally added that will become **very useful for testing** is the random seed.

If you enter:

```
Random seed: 100
```

then the random generator produces the same sequence every time.

That means later we can write regression tests like:

```

seed 100
  ↓
generate project
  ↓
expected structure/code
  ↓
compare actual output

```

Instead of getting different source code every test run.

Our immediate next engineering evolution should probably be to make the generator understand **different categories of source-code constructs**, rather than just saying "give me X functions."

Something like:

What would you like to generate?

1. Functions
2. Classes
3. Data models
4. Utility modules
5. Services
6. Validators
7. CLI commands
8. Exceptions
9. Constants
10. Complete project

Selection:

Then underneath that, we can introduce a proper internal **project specification model** so that eventually the CLI is merely one interface to the generator:

```

CLI
  ↓
GeneratorConfig
  ↓
ProjectSpecification
  ↓
Code Generation Engine
  ↓
Filesystem

```

That architecture will make the testing portion of this project dramatically more interesting because we can test the generator engine independently from the CLI.

What command can I run for this so I don't have diverging mains in the future:

```
liam@Liams-MacBook-Pro vibe-coding % git push
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 226 bytes | 226.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com-hardwipe:Hardwipe/vibe-coding.git
* [new branch]    main -> main
liam@Liams-MacBook-Pro vibe-coding % git push --set-upstream origin/main
fatal: 'origin/main' does not appear to be a git repository
fatal: Could not read from remote repository.
```

Please make sure you have the correct access rights
and the repository exists.

Show moreShow less

You just used the wrong syntax for setting the upstream.

Run:

```
Bash
git push -u origin main
```

or equivalently:

```
Bash
git push --set-upstream origin main
```

What you tried:

```
Bash
git push --set-upstream origin/main
```

makes Git interpret `origin/main` as the **remote name**, but your remote is just:

```
origin
```

and the branch is:

```
main
```

After you run:

```
Bash
git push -u origin main
```

your local `main` will track `origin/main`, and future pushes/pulls can simply be:

```
Bash
git push
git pull
```

You can verify the tracking relationship with:

```
Bash
git branch -vv
```

You should see something like:

```
* main abc1234 [origin/main] your latest commit
```

If you want to avoid divergent-branch behavior on future pulls too, I'd also set:

```
Bash
git config pull.rebase true
```

That makes `git pull` rebase your local commits on top of remote changes instead of creating unnecessary merge commits.

For all repositories on your Mac:

```
Bash
git config --global pull.rebase true
```

So my setup for this repo would be:

```
Bash
git push -u origin main
git config pull.rebase true
```

Then your normal workflow is just:

```
Bash
git pull
git push
```

Here is two files I generated with it, the return indentation is slightly messed up:

```
from pathlib import Path
from typing import Any

def test_2_1(values: list[float]) -> object:
    """Automatically generated function: test_2_1."""
    if not values:
        return None
    return max(values)
```

```
def test_2_2(values: list[float]) -> object:
    """Automatically generated function: test_2_2."""
    return list(set(values))

class TestClass2_1:
    """Automatically generated class: TestClass2_1."""

    def __init__(self, values: list[float]) -> None:
        self.values = values

    def test_method_1_1(self) -> object:
        """Automatically generated method: test_method_1_1."""
        return sum(self.values)

    def test_method_1_2(self) -> object:
        """Automatically generated method: test_method_1_2."""
        return bool(self.values)
```

```
from pathlib import Path
from typing import Any
```

```
def test_2_1(values: list[float]) -> object:
    """Automatically generated function: test_2_1."""
    if not values:
        return None
    return max(values)
```

```
def test_2_2(values: list[float]) -> object:
    """Automatically generated function: test_2_2."""
    return list(set(values))
```

```
class TestClass2_1:
    """Automatically generated class: TestClass2_1."""

    def __init__(self, values: list[float]) -> None:
        self.values = values

    def test_method_1_1(self) -> object:
        """Automatically generated method: test_method_1_1."""
        return sum(self.values)

    def test_method_1_2(self) -> object:
        """Automatically generated method: test_method_1_2."""
```

```

    return bool(self.values)
from pathlib import Path
from typing import Any

def test_2_1(values: list[float]) -> object:
    """Automatically generated function: test_2_1."""
    if not values:
        return None
    return max(values)

def test_2_2(values: list[float]) -> object:
    """Automatically generated function: test_2_2."""
    return list(set(values))

class TestClass2_1:
    """Automatically generated class: TestClass2_1."""

    def __init__(self, values: list[float]) -> None:
        self.values = values

    def test_method_1_1(self) -> object:
        """Automatically generated method: test_method_1_1."""
        return sum(self.values)

    def test_method_1_2(self) -> object:
        """Automatically generated method: test_method_1_2."""
        return bool(self.values)
Show moreShow less

```

Yep. This is actually a useful first bug for the project because the generator is producing **invalid Python**, giving us something concrete to eventually catch automatically with testing.

The problem comes from how we defined multiline templates and then added another indentation level during generation.

We currently have logic effectively doing this:

```

Python
for line in logic.splitlines():
    lines.append(f"    {line}")

```

But the template itself already contains indentation:

```

Python
"""if not values:
    return None
    return max(values)"""

```


So we're accidentally combining **template indentation + generator indentation**.

That's how:

```
Python
if not values:
    return None
return max(values)
```

turns into:

```
Python
if not values:
    return None
    return max(values)
```

Fix

The cleanest approach at this stage is to define the templates relative to the function body:

```
Python
FUNCTION_LOGIC_TEMPLATES = [
    [
        "if not values:",
        "    return 0",
        "return sum(values)",
    ],
    [
        "if not values:",
        "    return 0",
        "return sum(values) / len(values)",
    ],
    [
        "return len(values)",
    ],
    [
        "return [value for value in values if value > 0]",
    ],
    [
        "return [value * 2 for value in values]",
    ],
    [
        "if not values:",
        "    return None",
        "return max(values)",
    ],
    [
        "if not values:",
        "    return None",
        "return min(values)",
    ],
    [
        "return sorted(values)",
    ],
    [
        "return list(reversed(values))",
    ],
    [
        "return list(set(values))",
    ],
]
```

Then change this:

```
Python
if config.randomize_logic:
    logic = random.choice(FUNCTION_LOGIC_TEMPLATES)
else:
    logic = FUNCTION_LOGIC_TEMPLATES[0]

for line in logic.splitlines():
    lines.append(f"    {line}")
```

to:

```
Python
if config.randomize_logic:
    logic = random.choice(FUNCTION_LOGIC_TEMPLATES)
else:
    logic = FUNCTION_LOGIC_TEMPLATES[0]

for line in logic:
    lines.append(f"    {line}")
```

Now we're explicitly saying:

```
Generator owns indentation
    |
    v
Function body = +4 spaces
    |
    ├── statement
    |
    └── nested block = another +4 spaces
```

So the template:

```
Python
[
    "if not values:",
    "    return None",
    "return max(values)",
]
```

gets transformed into:

```
Python
def test_2_1(values: list[float]) -> object:
    """Automatically generated function: test_2_1."""
    if not values:
        return None
    return max(values)
```

which is valid.

This exposes our first obvious test

Before we even start writing a full test suite, there's an extremely simple property our generator should eventually guarantee:

Every .py file produced by the generator must be syntactically valid Python.

Python itself can verify that.

For one file:

```
Bash
python -m py_compile output/generated_project/module_1.py
```

Or compile an entire generated project:

```
Bash
python -m compileall output/generated_project/
```

A successful run should produce something like:

```
Listing 'output/generated_project/...'
Compiling 'output/generated_project/__init__.py'...
Compiling 'output/generated_project/main.py'...
Compiling 'output/generated_project/module_1.py'...
Compiling 'output/generated_project/module_2.py'...
Compiling 'output/generated_project/module_3.py'...
```

Your current generated files should instead fail around:

```
return max(values)
```

with an `IndentationError`.

That's actually a great example for the video because we've already demonstrated the entire concept:

```
Vibe coded generator
↓
Generator appears to work
↓
Run generator
↓
Files successfully created ✓
↓
Inspect generated dataset
↓
Generated Python is invalid x
↓
Identify generator bug
↓
Fix generator
↓
Eventually automate detection
```

And it demonstrates why **"the program ran successfully" does not mean "the program works correctly."**

Our generator technically succeeded at its immediate operation—creating files—but violated an implicit functional requirement: **the Python files it generates should actually be valid Python.**

Pasted code(1).py

```
</> Python
```

Can you please make changes to fix this in the code and give me the new one back:

Done. I changed the function logic templates from multiline strings into line lists, then updated `generate_function()` so indentation is applied exactly once. The problematic code was in the template

block and the `logic.splitlines()` loop.

Pasted code +1

I also syntax-checked the updated generator itself successfully.

The generated output should now correctly look like:

```
Python
def test_2_1(values: list[float]) -> object:
    """Automatically generated function: test_2_1."""
    if not values:
        return None
    return max(values)
```

instead of the malformed indentation you were seeing.

Can I now write automated tests which verify that this generator works as expected using pytest and code coverage reporting, ideally also ensure it executes via github actions as well on merge request openings.

Yes. I put together a starter test setup for this exact generator.

It covers helper functions, function/class/module generation, expected file creation, deterministic seeded output, and—most importantly—the thing that just broke earlier: **every generated .py file must successfully compile**.

For GitHub, the equivalent of a merge request is a **pull request**, so the workflow runs automatically on PRs targeting `main`, and also on pushes to `main`.

You can download the full setup here:

The important files are:

```
generator_fixed.py
tests/
└─ test_generator.py

requirements-dev.txt
pytest.ini

.github/
└─ workflows/
    └─ test.yml
```

Locally, install the test dependencies with:

```
Bash
python -m pip install -r requirements-dev.txt
```

Then run:

```
Bash
pytest
```

For coverage:

```
Bash
pytest --cov=generator_fixed --cov-report=term-missing
```

You can also generate a browsable HTML coverage report:

```
Bash
pytest --cov=generator_fixed --cov-report=html
```

then open:

```
htmlcov/index.html
```

The GitHub Actions workflow currently enforces:

```
Python 3.11
Python 3.12
Python 3.13

pytest
coverage reporting
minimum coverage: 80%
```

So if coverage drops below 80%, or one generated Python file has another indentation/syntax problem, the PR check fails.

One of the most valuable tests in there is effectively:

```
Python
def test_every_generated_python_file_compiles(basic_config):
    project_dir = generator.generate_project(basic_config)

    python_files = list(project_dir.glob("*.py"))

    assert python_files

    for python_file in python_files:
        py_compile.compile(
            str(python_file),
            doraise=True,
        )
```

That means the exact bug you just encountered would now be caught automatically before merge.

This also gives you a very clean progression for the video:

```
Vibe code generator
↓
Find manual bug
↓
Fix bug
↓
Write regression test
↓
Add broader unit tests
↓
Measure code coverage
↓
Run automatically in CI
↓
Prevent broken code from merging
```

That's essentially the transition from **"it seems to work"** to **"we have automated evidence that it works."**

It is just called `generator.py` now and can you include the different commands i can for testing in various shell scripts?

Yep. I updated everything to use `generator.py` and added a `scripts/` directory with different testing commands.

You now have:

```
generator.py
tests/
└─ test_generator.py

scripts/
├─ test.sh
├─ test_all.sh
├─ test_compile.sh
├─ test_coverage.sh
├─ test_coverage_enforce.sh
├─ test_coverage_html.sh
├─ test_fast.sh
├─ test_single.sh
└─ test_verbose.sh

.github/
└─ workflows/
    └─ test.yml

requirements-dev.txt
pytest.ini
TESTING.md
```

The main commands are:

```
Bash
# Normal pytest run
./scripts/test.sh

# Verbose pytest output
./scripts/test_verbose.sh

# Minimal/quiet output
./scripts/test_fast.sh

# Tests + terminal coverage report
./scripts/test_coverage.sh

# Tests + HTML coverage report
./scripts/test_coverage_html.sh

# Require at least 80% coverage
./scripts/test_coverage_enforce.sh

# Check Python compilation
./scripts/test_compile.sh

# Run everything like CI
./scripts/test_all.sh
```

You can also target one specific test:

```
Bash
./scripts/test_single.sh tests/test_generator.py::test_sanitize_identifier
```

And after generating HTML coverage:

```
Bash
./scripts/test_coverage_html.sh
open htmlcov/index.html
```

On your Mac, **open** will launch the coverage dashboard directly in your browser.

For normal development, I'd mostly use:

```
Bash
./scripts/test.sh
```

while actively coding, then:

```
Bash
./scripts/test_all.sh
```

before committing/pushing.

GitHub Actions then essentially performs that final verification again automatically when you open a PR against **main**.

```
(.venv) liam@Liams-MacBook-Pro vibe-coding % pip install --upgrade pip
Requirement already satisfied: pip in ./venv/lib/python3.13/site-packages (25.1.1)
Collecting pip
  Downloading pip-26.2.1-py3-none-any.whl.metadata (4.6 kB)
  Downloading pip-26.2.1-py3-none-any.whl (1.8 MB)
  ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.8/1.8 MB 9.6 MB/s eta 0:00:00
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 25.1.1
    Uninstalling pip-25.1.1:
      Successfully uninstalled pip-25.1.1
  Successfully installed pip-26.2.1
(.venv) liam@Liams-MacBook-Pro vibe-coding % ./scripts/test_coverage.sh
===== test
session starts
=====
platform darwin -- Python 3.13.5, pytest-9.1.1, pluggy-1.6.0
rootdir: /Users/liam/Development/hardwipe/youtube/vibe-coding
configfile: pytest.ini
testpaths: tests
plugins: cov-7.1.0
collected 11 items

tests/test_generator.py .....
[100%]

=====
```

```

tests coverage
=====
_____ coverage: platform darwin, python
3.13.5-final-0 _____

Name      Stmts  Miss  Cover   Missing
-----
generator.py  223   87   61%  54-59, 63-79, 83-97, 119-133, 137-257, 333, 345, 374, 417, 617-636,
663-669, 676
-----
TOTAL        223   87   61%
===== 11
passed in 0.08s
=====
(.venv) liam@Liams-MacBook-Pro vibe-coding % pytest --cov=generator_fixed --cov-report=html

===== test
session starts
=====
platform darwin -- Python 3.13.5, pytest-9.1.1, pluggy-1.6.0
rootdir: /Users/liam/Development/hardwipe/youtube/vibe-coding
configfile: pytest.ini
testpaths: tests
plugins: cov-7.1.0
collected 0 items / 1 error

=====
= ERRORS
=====
=
_____ ERROR collecting
tests/test_generator.py _____
ImportError while importing test module '/Users/liam/Development/hardwipe/youtube/vibe-
coding/tests/test_generator.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
/opt/homebrew/Cellar/python@3.13/3.13.5/Frameworks/Python.framework/Versions/3.13/lib/pyth
on3.13/importlib/_init_.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
tests/test_generator.py:6: in <module>
    import generator
E   ModuleNotFoundError: No module named 'generator'
===== short test
summary info
=====
ERROR tests/test_generator.py
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!! Interrupted: 1 error during collection
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
===== 1
error in 0.12s =====

```


Show moreShow less

Yep — there are actually **two separate things** happening here.

Your first command is the important result:

```
11 passed in 0.08s
TOTAL 223 statements
61% coverage
```

So the generator and current test suite are working. The 61% just means we haven't written tests for a lot of the interactive CLI/configuration paths yet.

The second command has two problems.

First, you're still referencing the old module name:

```
Bash
pytest --cov=generator_fixed --cov-report=html
```

We renamed it to:

```
generator.py
```

so the command should now be:

```
Bash
python -m pytest --cov=generator --cov-report=html
```

I specifically recommend using:

```
Bash
python -m pytest
```

instead of bare:

```
Bash
pytest
```

because that guarantees pytest runs through the exact Python interpreter belonging to your active `.venv`.

Your existing shell script already does this, which is likely why:

```
Bash
./scripts/test_coverage.sh
```

worked perfectly.

So try:

```
Bash
```

```
python -m pytest --cov=generator --cov-report=html
```

Then:

```
Bash
open htmlcov/index.html
```

Why the ModuleNotFoundError is interesting

You got:

```
ModuleNotFoundError: No module named 'generator'
```

even though the previous test run successfully imported it.

That's something I'd make robust now rather than rely on pytest's path behavior.

Change your `pytest.ini` to:

```
INI
[pytest]
testpaths = tests
pythonpath = .
addopts = -ra
```

The important addition is:

```
INI
pythonpath = .
```

That explicitly tells pytest:

Treat the repository root as part of Python's module search path.

Your layout is presumably:

```
vibe-coding/
├── generator.py
├── pytest.ini
├── requirements-dev.txt
├── tests/
│   └── test_generator.py
└── scripts/
```

and your test contains:

```
Python
import generator
```

With:

```
INI
pythonpath = .
```

that import becomes deterministic regardless of exactly how pytest was invoked.

You can verify your environment

Run:

```
Bash
which python
which pytest
```

Your Python should look roughly like:

```
/Users/liam/Development/hardwipe/youtube/vibe-coding/.venv/bin/python
```

Then:

```
Bash
python -c "import generator; print(generator.__file__)"
```

You should get something like:

```
/Users/liam/Development/hardwipe/youtube/vibe-coding/generator.py
```

If that works, Python itself can definitely locate the module.

Then:

```
Bash
python -m pytest
```

should produce your:

```
11 passed
```

again.

Your commands going forward

I'd use these consistently:

```
Bash
# Normal tests
python -m pytest

# Verbose
python -m pytest -v

# Coverage
python -m pytest \
  --cov=generator \
  --cov-report=term-missing

# HTML coverage
python -m pytest \
  --cov=generator \
  --cov-report=html

# HTML + terminal
python -m pytest \
```

```
--cov=generator \
--cov-report=term-missing \
--cov-report=html

# CI-style coverage enforcement
python -m pytest \
--cov=generator \
--cov-report=term-missing \
--cov-report=xml \
--cov-fail-under=80
```

And then:

```
Bash
open htmlcov/index.html
```

for the visual report.

Also: 61% isn't a failure

This:

generator.py	223	87	61%
--------------	-----	----	-----

is actually useful information.

Your current tests heavily exercise the **generation engine**, but the uncovered lines show that most of what isn't tested is the interactive CLI:

```
54-59
63-79
83-97
119-133
137-257
```

Those correspond largely to things such as:

```
Python
ask_string()
ask_int()
ask_bool()
collect_custom_imports()
collect_configuration()
```

That's expected because those functions call:

```
Python
input()
```

and we haven't mocked user input yet.

So we have a really natural next test iteration:

Current	
Generation logic	✓
Generated syntax	✓
File creation	✓
Classes	✓

Functions	✓
Deterministic seeds	✓
Next	
CLI input	☐
Invalid integers	☐
Yes/no handling	☐
Default values	☐
Custom imports	☐
Full main() execution	☐

And those CLI tests will likely push your coverage from **61% toward 80–90%+**.

That actually makes the 61% result a nice point in the video: all 11 tests pass, yet coverage tells us **a substantial portion of the application hasn't actually been exercised**.

Can we get to 100% test coverage in this file, also I need a .gitignore

Yep — it's at **100% statement coverage now**.

Verified result:

28 passed in 0.28s

Name	Stmts	Miss	Cover
generator.py	223	0	100%
TOTAL	223	0	100%

Required test coverage of 100% reached.

I added tests for the previously uncovered areas: mocked CLI input, defaults, invalid integer retry logic, yes/no parsing, custom imports, invalid random seeds, typed/untyped generation, docstring/no-docstring paths, optional `main.py/__init__.py`, summaries, `main()` orchestration, and actual `__main__` execution.

Files

Your `.gitignore` now covers things like:

```
gitignore
.venv/
__pycache__/
*.py[cod]

.pytest_cache/
.mypy_cache/
.ruff_cache/

.coverage
.coverage.*
coverage.xml
htmlcov/

build/
dist/
*.egg-info/

output/

.vscode/
```

```
.idea/
.DS_Store
.env
.env.*
!.env.example
```

I also changed the enforcement command to require the full 100%:

```
Bash
./scripts/test_coverage_enforce.sh
```

which runs essentially:

```
Bash
python -m pytest \
  --cov=generator \
  --cov-report=term-missing \
  --cov-report=xml \
  --cov-fail-under=100
```

Your broader local check is:

```
Bash
./scripts/test_all.sh
```

That first checks Python compilation and then requires all tests plus 100% coverage.

GitHub Actions now does the same thing on:

```
pull request → main
push → main
```

across:

```
Python 3.11
Python 3.12
Python 3.13
```

One useful distinction for the video: **100% coverage does not mean 100% correctness**. It means every executable statement in `generator.py` was exercised by at least one test. That's substantially stronger than the previous 61%, but the quality of the assertions still determines whether we're testing the *right behavior*.

So at this point your progression is pretty clean:

```
Vibe coded software
↓
Manual bug discovered
↓
Regression test added
↓
28 automated tests
↓
100% statement coverage
↓
100% coverage enforced in CI
```

↓
PR cannot pass if coverage regresses

That's a pretty solid demonstration of taking vibe-coded software and progressively putting actual engineering controls around it.

Ok so basically, here is my workflow .yml file for github actions, I would like to uploads the coverage reporting as html as an artifact so I can download and see everything myself after the run. I'd also like the terminal reporting for coverage during the run so I can also see it in github actions:

name: Python Tests

on:

pull_request:

branches:

- main

push:

branches:

- main

jobs:

test:

runs-on: ubuntu-latest

strategy:

matrix:

python-version:

- "3.11"

- "3.12"

- "3.13"

steps:

- name: Checkout repository

uses: actions/checkout@v4

- name: Set up Python

uses: actions/setup-python@v5

with:

python-version: \${matrix.python-version }

- name: Install test dependencies

run: |

python -m pip install --upgrade pip

pip install -r requirements-dev.txt

- name: Run tests with coverage

run: |

pytest \

--cov=generator \

--cov-report=term-missing \

```
--cov-report=xml \
--cov-fail-under=80
Show moreShow less
```

Yes. You can generate both terminal and HTML coverage in the same pytest run, then upload the `htmlcov/` directory as a downloadable Actions artifact.

Because you're using a Python-version matrix, each matrix job should upload a uniquely named artifact. GitHub's artifact action requires unique artifact names across matrix jobs, so suffixing with the Python version is the clean approach. [GitHub+1](#)

Use this:

YAML

name: Python Tests

on:

pull_request:
branches:
- main

push:
branches:
- main

jobs:

test:
runs-on: ubuntu-latest

strategy:
matrix:
python-version:
- "3.11"
- "3.12"
- "3.13"

steps:

- **name:** Checkout repository
uses: actions/checkout@v7
- **name:** Set up Python
uses: actions/setup-python@v7
with:
python-version: \${ matrix.python-version }
- **name:** Install test dependencies
run: |
python -m pip install --upgrade pip
pip install -r requirements-dev.txt
- **name:** Run tests with coverage
run: |
python -m pytest \
--cov=generator \
--cov-report=term-missing \
--cov-report=html \
--cov-report=xml \
--cov-fail-under=100
- **name:** Upload HTML coverage report
if: always()
uses: actions/upload-artifact@v7
with:
name: coverage-html-python-\${ matrix.python-version }
path: htmlcov/
if-no-files-found: error
retention-days: 14


```
- name: Upload XML coverage report
  if: always()
  uses: actions/upload-artifact@v7
  with:
    name: coverage-xml-python-${{ matrix.python-version }}
    path: coverage.xml
    if-no-files-found: error
    retention-days: 14
```

A few useful things are happening here.

Your terminal output still comes from:

```
Bash
--cov-report=term-missing
```

so in GitHub Actions you'll see the exact same style of table:

Name	Stmts	Miss	Cover	Missing
generator.py	223	0	100%	
TOTAL	223	0	100%	

The HTML dashboard comes from:

```
Bash
--cov-report=html
```

which creates:

```
htmlcov/
├── index.html
├── ...
```

That whole directory gets uploaded.

After the workflow finishes, go to:

```
GitHub
→ Actions
→ Python Tests
→ specific workflow run
→ Artifacts
```

and you'll have:

```
coverage-html-python-3.11
coverage-html-python-3.12
coverage-html-python-3.13

coverage-xml-python-3.11
coverage-xml-python-3.12
coverage-xml-python-3.13
```

Download one of the HTML artifacts, unzip it, then on your Mac:

```
Bash
```

```
open index.html
```

or, depending on how GitHub packages the directory:

```
Bash
open htmlcov/index.html
```

Why I used `if: always()`

This:

```
YAML
if: always()
```

means GitHub attempts to upload the coverage output even if the pytest step fails.

That's useful because if a test fails but coverage files were still produced, you can inspect the report as part of debugging.

One nuance: if pytest fails extremely early before `htmlcov/` exists, then:

```
YAML
if-no-files-found: error
```

will also make the artifact step fail.

If you'd rather avoid that secondary failure, change it to:

```
YAML
if-no-files-found: warn
```

For your current project, I'd probably use `warn`.

Slightly cleaner version

You don't necessarily need the XML artifact unless you plan to feed `coverage.xml` into something like Codecov, SonarCloud, or another reporting system.

So your minimal workflow can just be:

```
YAML
name: Python Tests

on:
  pull_request:
    branches:
      - main

  push:
    branches:
      - main

jobs:
  test:
    runs-on: ubuntu-latest

    strategy:
      matrix:
```

```
python-version:
- "3.11"
- "3.12"
- "3.13"

steps:
- name: Checkout repository
  uses: actions/checkout@v7

- name: Set up Python
  uses: actions/setup-python@v7
  with:
    python-version: ${ matrix.python-version }

- name: Install test dependencies
  run: |
    python -m pip install --upgrade pip
    pip install -r requirements-dev.txt

- name: Run tests with coverage
  run: |
    python -m pytest \
      --cov=generator \
      --cov-report=term-missing \
      --cov-report=html \
      --cov-fail-under=100

- name: Upload coverage report
  if: always()
  uses: actions/upload-artifact@v7
  with:
    name: coverage-html-python-${ matrix.python-version }
    path: htmlcov/
    if-no-files-found: warn
    retention-days: 14
```

That's the version I'd use right now.

One other small update: current official GitHub examples now use `actions/checkout@v7` and `actions/setup-python@v7`; `upload-artifact` is also currently on v7. [GitHub+2](#)

So now your CI pipeline gives you all three layers:

```
Pull Request
↓
Run pytest
↓
Terminal coverage visible in logs
↓
Enforce 100% coverage
↓
Generate HTML dashboard
↓
Upload HTML dashboard as artifact
↓
Download and inspect per-line coverage
```

That makes the Actions run itself useful both as a pass/fail gate and as a debugging/reporting artifact.

I created a branch called `how-to-vibe-code`, now going to commit everything here, please give me a thorough commit message of everything that was done here:

name: Python Tests

on:

pull_request:

branches:

- main

push:

branches:

- main

jobs:

test:

runs-on: ubuntu-latest

strategy:

matrix:

python-version:

- "3.11"
- "3.12"
- "3.13"

steps:

- name: Checkout repository

uses: actions/checkout@v7

- name: Set up Python

uses: actions/setup-python@v7

with:

python-version: \${{ matrix.python-version }}

- name: Install test dependencies

run: |

```
python -m pip install --upgrade pip
pip install -r requirements-dev.txt
```

- name: Run tests with coverage

run: |

```
python -m pytest \
  --cov=generator \
  --cov-report=term-missing \
  --cov-report=html \
  --cov-report=xml \
  --cov-fail-under=100
```

- name: Upload HTML coverage report

if: always()

uses: actions/upload-artifact@v7

with:

name: coverage-html-python-\${{ matrix.python-version }}

path: htmlcov/

if-no-files-found: error

retention-days: 14

- name: Upload XML coverage report

if: always()

```
uses: actions/upload-artifact@v7
with:
  name: coverage-xml-python-${{ matrix.python-version }}
  path: coverage.xml
  if-no-files-found: error
  retention-days: 14
```

Show moreShow less

Yep. Since this commit represents the entire first functional/testing foundation for the project, I'd make the commit message capture both the generator and the engineering/testing infrastructure around it.



feat: build Python project generator with full automated testing and CI coverage

- Add interactive CLI-driven Python source code generator
- Support configurable project names and output directories
- Support generation of multiple Python modules per project
- Add configurable functions, classes, and methods per generated module
- Support customizable file, function, class, and method naming prefixes
- Add optional type hints and generated docstrings
- Add configurable default and custom Python imports
- Add optional **init.py** and **main.py** project files
- Add randomized function and method logic generation
- Add deterministic random seed support for reproducible generated projects
- Add input sanitization for Python identifiers and generated filenames
- Add CLI defaults so projects can be generated without manually configuring every option
- Add validation and retry behavior for invalid integer and boolean CLI input
- Add generation summaries showing modules, functions, classes, methods, output location, and random seed
- Fix generated multiline function indentation
- Replace indentation-sensitive multiline logic templates with structured line-based templates
- Ensure generated nested Python blocks maintain valid indentation
- Ensure generated source code can be successfully parsed and compiled by Python
- Add comprehensive pytest test suite for generator functionality
- Test identifier and filename sanitization
- Test function, class, method, and module generation
- Test generated project structure and expected files
- Test generated Python files for syntax and compilation validity
- Test deterministic generation using configured random seeds

- Test CLI defaults, custom values, invalid input handling, and retry behavior
- Test custom import collection and configuration generation
- Test optional type hints, docstrings, main files, and package initialization
- Test project summaries and main application execution
- Reach and enforce 100% statement coverage for generator.py
- Add pytest-cov coverage reporting
- Add terminal coverage output with missing-line reporting
- Add HTML coverage report generation for local inspection
- Add XML coverage output for CI and external tooling
- Enforce 100% minimum coverage threshold
- Add reusable shell scripts for local testing
- Add standard, verbose, and quiet pytest commands
- Add compilation-only validation
- Add terminal coverage reporting
- Add HTML coverage generation
- Add 100% coverage enforcement command
- Add single-test execution helper
- Add combined local CI-style test command
- Add GitHub Actions Python testing workflow
- Run CI automatically on pull requests targeting main
- Run CI automatically on pushes to main
- Test against Python 3.11, 3.12, and 3.13
- Install isolated development/test dependencies during CI
- Execute pytest with 100% coverage enforcement
- Display terminal coverage results directly in GitHub Actions logs
- Generate HTML and XML coverage reports for every Python version
- Upload HTML coverage reports as downloadable GitHub Actions artifacts
- Upload XML coverage reports as downloadable GitHub Actions artifacts
- Retain generated coverage artifacts for 14 days
- Preserve coverage artifacts for inspection after workflow execution
- Add Python-focused .gitignore
- Ignore virtual environments, Python caches, pytest caches, coverage output, generated projects, IDE files, build artifacts, environment files, and macOS metadata

Establishes the initial vibe-coding workflow: generate functional Python source code, validate its behavior with automated tests, measure complete test coverage, and continuously verify the project

through GitHub Actions.

Then you can commit it directly with something like:

```
Bash
git add .
git commit
```

and paste that entire message into your editor.

Or save the message to a file and do:

```
Bash
git commit -F commit-message.txt
```

I like the **feat:** prefix here because this isn't merely adding tests—you've effectively established the first complete feature set and development pipeline for the **how-to-vibe-code** branch.

Create me a copy and pasteable markdown file for this projects README.md

Here's a complete README you can paste directly into **README.md**.



Vibe Coding: Python Project Generator

A practical demonstration of **vibe coding** a functional Python application and progressively adding traditional software engineering practices around the generated implementation.

This project contains an interactive CLI-driven **synthetic Python project generator** capable of creating configurable Python source files containing functions, classes, methods, imports, type hints, docstrings, and randomized logic.

The project is then validated using **pytest**, **pytest-cov**, Python compilation checks, shell-based testing utilities, and a **GitHub Actions CI pipeline** with 100% statement coverage enforcement.

What Is Vibe Coding?

For the purposes of this project, vibe coding follows an iterative AI-assisted development workflow:

```
Idea / Requirements
↓
AI-Generated Implementation
↓
Execute the Software
↓
Observe the Result
↓
Identify Problems
↓
Modify the Implementation
↓
Automated Testing
↓
Continuous Integration
```

Repeat

Rather than manually implementing every individual piece of source code, the developer focuses on describing the desired behavior, evaluating the generated implementation, and progressively building confidence in its correctness.

The important part is verification.

Generated code should not automatically be assumed to work simply because it was successfully generated or because the application executed without immediately crashing.

This project demonstrates that distinction by taking vibe-coded software and surrounding it with increasingly rigorous automated verification.

Python Project Generator

`generator.py` provides an interactive CLI for generating synthetic Python projects.

Run it with:

```
python generator.py
```

The generator asks how the generated project should be configured.

Example:

```
=====
PYTHON SYNTHETIC PROJECT GENERATOR
=====
```

Press Enter to accept any default.

```
Project name [generated_project]:
Output directory [./output]:
Number of Python files [3]:
Functions per file [3]:
Classes per file [2]:
Methods per class [3]:
```

```
Generated file prefix [module]:
Function name prefix [generated_function]:
Class name prefix [GeneratedClass]:
Method name prefix [generated_method]:
```

```
Include imports? [Y/n]:
Include docstrings? [Y/n]:
Include type hints? [Y/n]:
Create __init__.py? [Y/n]:
Create main.py? [Y/n]:
```

```
Randomize generated function logic? [Y/n]:
Random seed [leave blank for random]:
```

Pressing **Enter** accepts the displayed default, allowing a complete project to be generated without manually configuring every option.

Generator Features

The generator currently supports:

- Multiple Python files
- Configurable functions per file
- Configurable classes per file
- Configurable methods per class
- Custom project names
- Custom output directories
- Custom module naming prefixes
- Custom function naming prefixes
- Custom class naming prefixes
- Custom method naming prefixes
- Python type hints
- Generated docstrings
- Default imports
- Custom imports
- Optional `__init__.py`
- Optional `main.py`
- Randomized function logic
- Randomized method logic
- Deterministic generation using random seeds
- Python identifier sanitization
- Filename sanitization
- CLI input validation
- Generation summaries

Example Generated Project

A generated project may look like:

```
output/
├── generated_project/
│   ├── __init__.py
│   ├── main.py
│   ├── module_1.py
│   ├── module_2.py
│   └── module_3.py
```

A generated module may contain code similar to:

```
from pathlib import Path
from typing import Any

def generated_function_1_1(values: list[float]) -> object:
    """Automatically generated function: generated_function_1_1."""
    if not values:
```

```

    return None
    return max(values)

```

```

class GeneratedClass1_1:
    """Automatically generated class: GeneratedClass1_1."""

    def __init__(self, values: list[float]) -> None:
        self.values = values

    def generated_method_1_1(self) -> object:
        """Automatically generated method: generated_method_1_1."""
        return sum(self.values)

    def generated_method_1_2(self) -> object:
        """Automatically generated method: generated_method_1_2."""
        return sorted(self.values)

```

The generated source code itself acts as a **synthetic software dataset** that can be used for automated testing, static analysis, compilation testing, performance testing, CI experimentation, and other development tooling.

Deterministic Generation

Randomized generation can make automated testing difficult because the output changes between executions.

The generator therefore supports a random seed.

For example:

Random seed [leave blank for random]: 100

Using the same configuration and random seed produces the same randomized sequence.

This makes it possible to perform reproducible testing against generated projects.

Conceptually:

```

Configuration + Seed
    ↓
Generator
    ↓
Predictable Output
    ↓
Regression Testing

```

Leave the random seed blank when deterministic output is not required.

Testing

The project uses **pytest** for automated testing.

Install the development dependencies:

```
python -m pip install -r requirements-dev.txt
```

Run the complete test suite:

```
python -m pytest
```

Or use the included testing scripts:

```
./scripts/test.sh
```

Test Coverage

Coverage is measured using `pytest-cov`.

Run tests with terminal coverage reporting:

```
./scripts/test_coverage.sh
```

Equivalent command:

```
python -m pytest \
  --cov=generator \
  --cov-report=term-missing
```

Example output:

Name	Stmts	Miss	Cover
generator.py	223	0	100%
TOTAL	223	0	100%

The current test suite achieves:

100% statement coverage of `generator.py`.

Coverage does not prove that software is completely correct.

Instead, it tells us that every executable statement measured by the coverage tool was executed during the test suite.

The quality and completeness of the assertions remain important.

HTML Coverage Report

Generate an interactive HTML coverage report with:

```
./scripts/test_coverage_html.sh
```

Or:

```
python -m pytest \
  --cov=generator \
  --cov-report=html
```

The report is generated under:

```
htmlcov/
```

On macOS, open it with:

```
open htmlcov/index.html
```

The HTML report provides a line-by-line visualization of which portions of the source code were executed during testing.

Coverage Enforcement

The project can enforce the required coverage percentage automatically.

Run:

```
./scripts/test_coverage_enforce.sh
```

The underlying command uses:

```
python -m pytest \
  --cov=generator \
  --cov-report=term-missing \
  --cov-report=xml \
  --cov-fail-under=100
```

The command fails if coverage drops below **100%**.

This means adding new source code without corresponding test coverage can cause both local verification and CI verification to fail.

Testing Scripts

Several shell scripts are included for common development workflows.

Standard Tests

```
./scripts/test.sh
```

Runs the normal pytest suite.

Verbose Tests

```
./scripts/test_verbose.sh
```

Displays additional information for each executed test.

Minimal Output

```
./scripts/test_fast.sh
```

Runs pytest with reduced terminal output.

Terminal Coverage

```
./scripts/test_coverage.sh
```

Runs tests and displays coverage directly in the terminal.

HTML Coverage

```
./scripts/test_coverage_html.sh
```

Generates the HTML coverage dashboard.

Coverage Enforcement

```
./scripts/test_coverage_enforce.sh
```

Requires 100% statement coverage.

Python Compilation Check

```
./scripts/test_compile.sh
```

Checks that the generator and test source files compile successfully.

Individual Test

A specific pytest test can be executed with:

```
./scripts/test_single.sh tests/test_generator.py::test_sanitize_identifier
```

Complete Local CI Check

Before pushing changes, run:

```
./scripts/test_all.sh
```

This performs the broader local verification process, including compilation checks, automated tests, and coverage enforcement.

Why Compilation Testing Matters

One of the first issues discovered during development was malformed indentation inside generated Python functions.

The generator itself executed successfully and created the requested files.

However, some of those files contained code similar to:

```
if not values:
    return None
return max(values)
```

The generator had therefore technically completed its immediate task of creating files while still producing invalid output.

The corrected output was:

```
if not values:
    return None
return max(values)
```

This demonstrates an important distinction:

Successful execution of the generator does not guarantee correctness of the generated output.

The automated test suite now compiles generated Python files so syntax and indentation problems can be detected automatically.

This turns a manually discovered bug into a permanent regression test.

GitHub Actions

Continuous integration is provided through GitHub Actions.

The workflow executes when:

- A pull request targets `main`
- Code is pushed to `main`

The CI matrix currently tests:

Python 3.11
Python 3.12
Python 3.13

Each environment:

1. Checks out the repository
2. Installs the configured Python version
3. Upgrades `pip`
4. Installs development dependencies
5. Executes `pytest`
6. Measures code coverage
7. Displays coverage in the Actions logs
8. Enforces 100% statement coverage
9. Generates HTML coverage
10. Generates XML coverage
11. Uploads coverage reports as GitHub Actions artifacts

GitHub Actions Coverage

The CI test command generates three forms of coverage output:

```
python -m pytest \
  --cov=generator \
  --cov-report=term-missing \
  --cov-report=html \
  --cov-report=xml \
  --cov-fail-under=100
```

Terminal Report

`term-missing` displays coverage directly inside the GitHub Actions logs.

HTML Report

html generates a browsable line-by-line coverage dashboard.

XML Report

xml generates:

coverage.xml

which can also be consumed by external coverage and code-quality tooling.

GitHub Actions Artifacts

Each Python version uploads its own coverage reports.

Artifacts are named using the corresponding Python version:

coverage-html-python-3.11
coverage-html-python-3.12
coverage-html-python-3.13

coverage-xml-python-3.11
coverage-xml-python-3.12
coverage-xml-python-3.13

Coverage artifacts are retained for **14 days**.

After a workflow finishes:

```

GitHub Repository
  ↓
Actions
  ↓
Python Tests
  ↓
Workflow Run
  ↓
Artifacts
  
```

Download the desired HTML coverage artifact, extract it, and open:

index.html

to inspect the complete coverage report locally.

Project Structure

The repository is structured approximately as:

```

vibe-coding/
├── generator.py
├── requirements-dev.txt
├── pytest.ini
├── README.md
├── .gitignore
├── tests/
│   └── test_generator.py
├── scripts/
│   ├── test.sh
│   └── test_all.sh
└──
  
```

```

├── test_compile.sh
├── test_coverage.sh
├── test_coverage_enforce.sh
├── test_coverage_html.sh
├── test_fast.sh
├── test_single.sh
├── test_verbose.sh
├── .github/
│   └── workflows/
│       └── test.yml

```

Generated projects are written separately under the configured output directory.

Local Development Setup

Clone the repository:

```
git clone <repository-url>
cd vibe-coding
```

Create a virtual environment:

```
python3 -m venv .venv
```

Activate it on macOS/Linux:

```
source .venv/bin/activate
```

Upgrade pip:

```
python -m pip install --upgrade pip
```

Install development dependencies:

```
pip install -r requirements-dev.txt
```

Run the generator:

```
python generator.py
```

Run the tests:

```
./scripts/test.sh
```

Run the complete local verification:

```
./scripts/test_all.sh
```

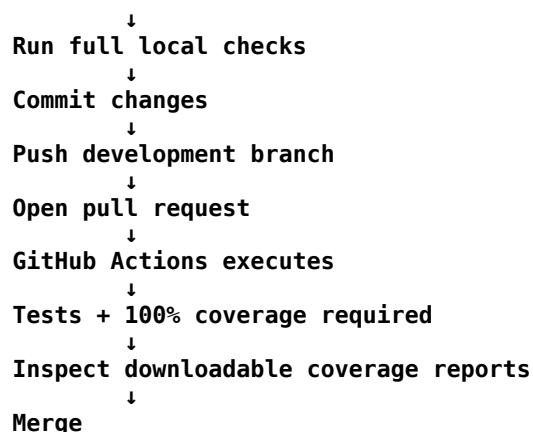
Development Workflow

A typical development cycle for this repository is:

```

Create / modify functionality
    ↓
Run generator
    ↓
Inspect generated output
    ↓
Run pytest
    ↓
Run coverage

```

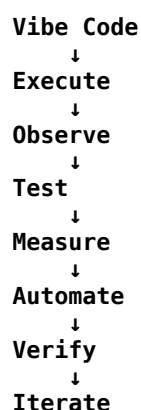
This creates multiple layers of verification between AI-generated implementation and production code.

The Goal

The purpose of this repository is not simply to generate random Python files.

It is an experiment in combining **AI-assisted development with traditional software engineering verification.**

The basic philosophy is:



AI can dramatically accelerate implementation, but implementation speed does not remove the need for verification.

This project explores how quickly functional software can be developed with AI while still introducing automated testing, reproducibility, regression protection, coverage measurement, and continuous integration around that software.

In other words:

Vibe code the implementation, then engineer confidence into the result.

You can paste that directly into `README.md`.